EmbedForge
PRP + Engineering Handoff Report

Version 1.0 — Implementation Handoff

1. Executive Summary

EmbedForge is an autonomous AI-assisted embedded engineering system. It accepts a human embedded-system requirement, researches missing technical information, requires human approval before external information becomes trusted project knowledge, generates firmware and an engineering plan, builds the firmware, analyzes actual software behavior, verifies that behavior against requirements and approved specifications, produces traceable evidence, repairs software defects, and finally reaches a human-controlled physical hardware validation stage.

The system is intentionally not a universal hardware simulator and does not claim to eliminate physical debugging. Physical hardware remains a validation boundary. When a human reports a physical failure, EmbedForge creates a debug session, gathers the observation as evidence, researches additional approved information when needed, analyzes the software/specification relationship, and either proposes/applies a software correction or classifies the issue as hardware-dependent/unknown when the available evidence is insufficient.

2. Non-Negotiable Design Principles

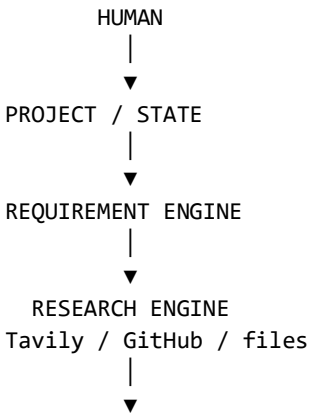
- Human authority: the human decides which discovered external resources become trusted engineering knowledge.
- Do not invent technical facts. Never silently manufacture an MCU, sensor model, address, frequency, pin, voltage, SDK, HAL, toolchain, register value, or protocol parameter.
- Requirement Engine captures and structures the user's request. It does not decide engineering facts.
- Research Engine discovers missing information. Tavily/GitHub are discovery mechanisms, not automatic truth sources.
- Human approval is required before discovered resources are promoted to trusted project knowledge.
- Nemotron is the reasoning brain, not the parser, compiler, verifier, or source of truth.
- Deterministic tools should perform parsing, building, static analysis, behavioral extraction, and rule-based verification whenever possible.
- Every important claim must have provenance/evidence.
- UNKNOWN and HARDWARE_DEPENDENT are valid outcomes. The system must not guess when evidence is insufficient.

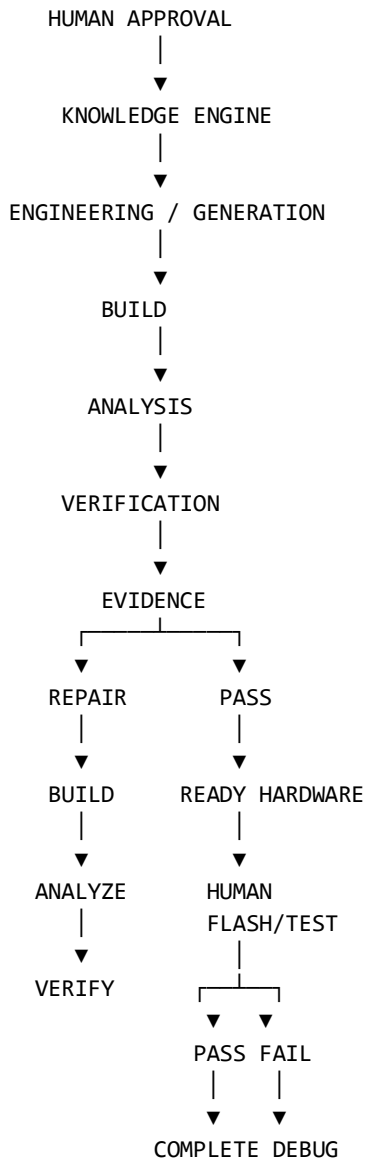
- Generated patches are never considered verified until rebuild → analysis → reverification succeeds.
- Artifacts are versioned and linked through lineage rather than silently overwritten.
- Physical flashing and physical testing are human-controlled.

3. Canonical Workflow

```
START
  ↓
UNDERSTANDING
  ↓
INFORMATION_CHECK
  ↓
RESEARCHING (when information is missing)
  ↓
RESOURCE_REVIEW
  ↓
KNOWLEDGE
  ↓
PLANNING
  ↓
GENERATING
  ↓
BUILD
  ↓
ANALYZE
  ↓
VERIFY
  ├── FAIL → REPAIR → BUILD → ANALYZE → VERIFY
  ├── UNKNOWN → REVIEW / HUMAN INPUT
  └── PASS → READY_FOR_HARDWARE
                        ↓
                  HUMAN FLASH/TEST
                  ├── PASS → COMPLETE
                  └── FAIL → DEBUGGING → RESEARCH/ANALYZE/VERIFY
```

4. System Architecture





5. Core Component Responsibilities

Project / State / Memory Engine: Owns project identity, workflow state, artifact registry, event history, version lineage, trust levels, and persistent engineering memory.

Requirement Engine: Captures only what the user explicitly requested. Structures functional/interface/timing/constraint information that is explicitly present and reports missing information without inventing it.

Research Engine: Detects information gaps, plans research, generates queries, calls Tavily/GitHub, collects and deduplicates results, classifies sources, extracts useful content, and determines whether gaps remain.

Human Approval Gateway: Shows discovered resources and requires the human to approve/reject them before they can become trusted project knowledge.

Knowledge Engine: Converts approved resources into structured, provenance-linked facts, constraints, register definitions, protocol information, timing rules, and other engineering knowledge. Preserves conflicts instead of silently overwriting them.

Engineering / Generation Engine: Creates an engineering plan, software architecture, interfaces/HAL boundaries, firmware, and tests from requirements plus approved knowledge.

Build Engine: Runs deterministic builds, records compiler/toolchain information and logs, and produces structured build evidence.

Analysis Engine: Determines actual software behavior using source parsing, AST/call graph/CFG/data-flow analysis, peripheral/register/protocol/timing/state analysis.

Verification Engine: Compares requirements + approved specifications + actual behavior. Uses deterministic rules first and classifies findings as PROVEN_MISMATCH, INFERRED_ISSUE, UNKNOWN/HARDWARE_DEPENDENT, or equivalent evidence states.

Evidence Engine: Builds traceable chains from requirement → specification → expected behavior → source/actual behavior → comparison → finding → patch.

Repair / Debug Engine: Performs root-cause reasoning, creates minimal patch plans, applies approved changes, rebuilds, reanalyzes, and reverifies.

Physical Validation Loop: Receives human test observations and treats them as evidence. It does not pretend that software-only analysis can prove physical hardware behavior.

6. Requirement Engine — Frozen Contract

The Requirement Engine is deliberately narrow. Its job is not to decide what the engineer should build. Its job is to represent what the human asked for and expose information that is absent.

```
User request
    ↓
Requirement parser
    ↓
Structured requirements
    ↓
Information-gap detector
    ↓
RequirementSet
    ├── source_text
    ├── requirements[]
    ├── information_gaps[]
    └── status
```

Examples of information that must never be invented by this component: sensor model, I2C address, MCU, board, pin mapping, sampling frequency, voltage, clock, SDK, HAL, compiler, register values, or initialization sequences. Those facts must come from the user or from

later approved resources.

7. Research Engine / Tavily

```
INFORMATION GAP
      ↓
RESEARCH PLANNER
      ↓
QUERY GENERATOR
      ↓
TAVILY / GITHUB
      ↓
RESULT COLLECTOR
      ↓
DEDUPLICATOR
      ↓
SOURCE CLASSIFIER
      ↓
CONTENT EXTRACTOR
      ↓
GAP ANALYZER
      ├── gaps remain → follow-up research
      └── sufficient → resource preview
                          ↓
                   HUMAN APPROVAL
                          ↓
                 APPROVED RESOURCES
```

- Prefer manufacturer datasheets/reference manuals and official documentation for authoritative specifications.
- Use GitHub/reference implementations for implementation patterns, not as automatic authoritative truth.
- Community/troubleshooting material may support investigation but should have a clearly lower trust classification.
- Preserve URL, title, source type, retrieval metadata, relevant section/page, and approval status.
- Research must have budgets: maximum searches, result count, depth, token/time budget, and stopping/relevance rules.
- Research can be invoked again during physical debugging.

8. Knowledge Model

```
KnowledgeItem
├── subject
├── property
├── value
├── conditions
├── source
├── source_type
```

```
├── location
├── approval_status
├── trust_level
└── provenance
```

Conditional facts must remain conditional. Project-specific configuration must not overwrite universal source facts. If two approved sources disagree, store both claims and create a conflict instead of silently selecting one.

9. Engineering and Firmware Generation

```
REQUIREMENTS
+
APPROVED KNOWLEDGE
        ↓
ENGINEERING PLAN
        ↓
SOFTWARE ARCHITECTURE
        ↓
HAL / DRIVER BOUNDARIES
        ↓
FIRMWARE GENERATION
        ↓
BUILD
```

The architecture should separate application logic from MCU/platform-specific implementations where practical. This does not mean building a universal hardware adapter. Each supported target has its own concrete implementation boundary.

10. Build and Analysis

- Build success is not correctness.
- Record compiler, SDK/toolchain, command, environment, warnings, errors, and artifact identifiers.
- Analyze generated/source firmware independently of the LLM.
- Build analysis should produce a BehaviorModel with functions, calls, control flow, data flow, peripherals, registers, protocols, timing, states, interrupts, and dependencies.
- Every extracted behavior should point back to source provenance such as file and line.

11. Verification and Evidence

```
REQUIREMENT
     +
APPROVED SPECIFICATION
     ↓
EXPECTED BEHAVIOR
     +
```

```
ACTUAL BEHAVIOR MODEL
     ↓
VERIFICATION
     ↓
FINDING
     ↓
EVIDENCE CASE
```

A finding should be traceable to the exact requirement, approved specification, source location, actual behavior, comparison rule, and resulting conclusion. Deterministic rules should be preferred for concrete facts such as addresses, register values, bit fields, protocol configuration, timing, GPIO configuration, initialization order, data format, and error handling.

12. Repair / Debug

```
FINDING
  ↓
EVIDENCE CASE
  ↓
ROOT-CAUSE ANALYSIS
  ↓
PATCH PLAN
  ↓
POLICY / APPROVAL
  ↓
PATCH
  ↓
BUILD
  ↓
ANALYZE
  ↓
VERIFY
```

A patch is not successful merely because it compiles. The repaired artifact must be reanalyzed and reverifed. Root causes should be distinguished from symptoms, and insufficient evidence should trigger further research or a request for human observation rather than an invented fix.

13. Physical Hardware Boundary

- EmbedForge does not claim to simulate every physical hardware condition.
- The human owns physical flashing and test execution.
- Human observations are first-class evidence.
- A physical failure can trigger a new research/debug cycle.
- If software/specification evidence proves a mismatch, propose or apply a software correction.

- If evidence cannot distinguish software from hardware/environment, classify the issue as UNKNOWN or HARDWARE_DEPENDENT.
- Never manufacture physical observations.

14. Tool Runtime and Permissions

```
NEMOTRON
   ↓
TOOL REQUEST
   ↓
POLICY / PERMISSION
   ↓
TOOL RUNTIME
   ↓
TOOL
   ↓
STRUCTURED RESULT
   ↓
PROJECT STATE / EVIDENCE
```

- Research stage: search/read tools allowed; modification/build/flash blocked.
- Planning: knowledge/read/plan generation allowed; physical actions blocked.
- Generation: source creation/modification/build allowed; flash blocked.
- Verification: read/analyze/verify allowed; modification/flash blocked.
- Repair: patch generation/application/build/analyze/verify allowed; physical flashing remains human-controlled.
- Every tool invocation should have structured arguments, structured results, failure states, and provenance.

15. Core Artifact Lineage

```
REQ-001
  ↓
RES-001
  ↓
KNOW-001
  ↓
PLAN-001
  ↓
FW-001
  ↓
BUILD-001
  ↓
BEHAV-001
  ↓
FIND-001
  ↓
CASE-001
  ↓
PATCH-001
```

```
  ↓
FW-002
  ↓
BUILD-002
  ↓
BEHAV-002
  ↓
VERIFY-002
```

16. Evaluation Strategy

- Use deterministic fixture projects with injected known defects.
- Fixtures should cover wrong I2C address, wrong sampling rate, wrong register value, wrong initialization order, wrong scaling, wrong byte order, missing timeout, and hardware-dependent unknown failure.
- Measure detection rate, false positives, classification correctness, patch generation, patch correctness, rebuild success, reverification success, regression rate, requirement coverage, evidence coverage, time, AI iterations, builds, and physical flash/debug cycles avoided.
- Do not publish performance claims until measured on controlled evaluation fixtures.

17. Recommended Repository Structure

```
ai-embedded-engineer/
├── embedforge/
│   ├── core/
│   ├── engines/
│   │   ├── requirements/
│   │   ├── research/
│   │   ├── knowledge/
│   │   ├── engineering/
│   │   ├── build/
│   │   ├── analysis/
│   │   ├── verification/
│   │   ├── evidence/
│   │   └── repair/
│   ├── models/
│   ├── runtime/
│   └── policy/
├── agent/
├── analysis/
├── backend/
├── evidence/
├── firmware/
├── frontend/
├── knowledge/
├── projects/
├── tests/
├── tools/
└── verification/
```

18. Implementation Order

1. Foundation / project lifecycle — completed.

2. Core data contracts — completed.

3. Requirement Engine — implement/finalize and freeze.

4. Research Engine abstraction and research-result contract.

5. Tavily integration.

6. Resource preview + human approval gateway.

7. Knowledge Engine and provenance.

8. Engineering planner.

9. Firmware generation.

10. Build Engine.

11. Behavior Analysis Engine.

12. Verification Engine.

13. Evidence Engine.

14. Repair/Debug Engine.

15. Physical validation/debug loop.

16. Evaluation fixtures and benchmark harness.

17. NeMo Agent Toolkit orchestration and polished end-to-end UX.

19. Antigravity / Coding-Agent PRP

Paste the following as the implementation handoff prompt to Antigravity or another coding agent:

```
You are implementing EmbedForge, an AI-assisted embedded engineering system.

FIRST: read the existing repository, tests, current data contracts, and project
documentation. Do not replace the architecture with a new architecture. Do not invent
missing requirements.

CURRENT STATUS:
- Foundation v0.1 exists and tests pass.
- Core data contracts v0.2 exist and tests pass.
- Requirement Engine v0.3/v0.3.1 is the current implementation target.
- The architecture is frozen at the conceptual level.
```

PRIMARY RULE:
The human is the authority for requirements and for approval of external technical resources.
The Requirement Engine MUST NOT invent engineering facts.

DO NOT:
- hard-code example values such as 100 Hz, 0x68, STM32, MPU6050, 3.3 V, GPIO numbers, registers, or pin mappings;
- assume a sensor model from generic words;
- treat Tavily/GitHub results as trusted without human approval;
- let the LLM be the sole verifier;
- claim physical hardware behavior without human observation;
- mark a patch verified merely because it compiles.

REQUIREMENT ENGINE:
Input: arbitrary human embedded-system request.
Output:
- original source_text;
- structured requirements containing only information explicitly represented in the request;
- information gaps;
- workflow status.

The parser may identify functional/interface/timing/constraint categories when those facts are explicitly present. It must preserve the source wording/provenance.
If information is absent, represent it as an information gap. Do not fill it.

RESEARCH ENGINE:
Build after Requirement Engine.
Flow:
information gaps → research planner → query generation → Tavily/GitHub → collection → deduplication → source classification → extraction → gap analysis → resource preview → human approval.

Resource approval is mandatory before promotion to trusted knowledge.

KNOWLEDGE ENGINE:
Store approved facts with provenance:
source, source type, URL/file, page/section/line where available, conditions, approval status, trust level.
Preserve conflicting sources rather than silently resolving them.

ENGINEERING:
requirements + approved knowledge → engineering plan → software architecture → firmware/driver generation → build.

ANALYSIS:
Use deterministic analysis wherever possible:
AST, call graph, CFG, data flow, peripheral/register/protocol/timing/state analysis.
Produce a structured BehaviorModel with source provenance.

VERIFICATION:
Compare requirements + approved specifications against actual behavior.

```
Prefer deterministic rules for concrete facts.
Finding classifications must include:
- PROVEN_MISMATCH
- INFERRED_ISSUE
- UNKNOWN / HARDWARE_DEPENDENT

EVIDENCE:
Every finding should be traceable:
requirement → approved specification → expected behavior → source/actual behavior →
comparison → finding → patch.

REPAIR:
finding → evidence → root cause → minimal patch → build → analysis → reverification.
Never call a patch verified before reverification.

PHYSICAL VALIDATION:
Human flashes and tests hardware.
Human observations become evidence.
A hardware failure starts a debug session.
If evidence is insufficient, return UNKNOWN/HARDWARE_DEPENDENT rather than guessing.

IMPLEMENTATION STYLE:
- small modules;
- typed data contracts;
- deterministic components separated from LLM reasoning;
- unit tests for every stage;
- integration tests for artifact flow;
- no hidden state;
- preserve lineage/version history;
- structured logs;
- clear error states.

BEFORE MODIFYING CODE:
1. inspect repository;
2. identify current implementation;
3. run current tests;
4. implement only the next agreed stage;
5. add/update tests;
6. run full test suite;
7. report changed files, tests, failures, and next state.

Do not move to the next major stage until the current stage is tested and stable.
```

20. Definition of Done

- Requirement Engine passes tests without hard-coded engineering assumptions.
- Research Engine can represent information gaps and perform bounded searches.
- Discovered resources are shown for human approval before trusted use.
- Approved resources have provenance.
- Knowledge conflicts are preserved.
- Generated firmware can be built deterministically.

- Behavior can be extracted independently of Nemotron.
- Verification produces evidence-backed findings.
- Repairs are followed by rebuild, reanalysis, and reverification.
- Physical failures can enter a human-driven debug loop.
- Evaluation fixtures produce measurable results.

21. Final Engineering Position

EmbedForge should be implemented as an evidence-driven engineering pipeline, not as a chatbot that jumps from natural language directly to firmware. The LLM supplies reasoning and engineering interpretation; deterministic subsystems establish what the software actually contains and does; approved external resources establish what the hardware/documentation says; and human physical testing establishes what actually happened on the real device.